

INSTITUTE OF ENGINEERING & MANAGEMENT

NEWTOWN, KOLKATA

**Department of Electronics & Communication
Engineering**

Module 4: Project Report

Smart Chassis Vibration Analysis System

Submitted by: PARTHIBA SARKAR

Enrollment Number: 12023002002106

Registration Number: 304202300900493

Class Roll Number: B-33

1. Introduction

Smart Chassis Vibration Analysis System

Vehicle chassis vibration can provide useful information about abnormal mechanical conditions. In this project, an ESP32 is connected to an MPU6050 accelerometer and gyroscope to acquire motion data. The measured acceleration is used to calculate a vibration level, while FFT analysis is used to identify the dominant frequency component. When the vibration level crosses the programmed threshold, a red LED is switched on and a warning is printed in the serial monitor.

2. Objective

- To interface an MPU6050 sensor with an ESP32 using I2C.
- To read acceleration values from the X, Y and Z axes.
- To calculate a vibration level from the acceleration data.
- To apply FFT analysis and identify the dominant vibration frequency.
- To generate an abnormal-vibration warning when the threshold is exceeded.
- To use an LED as a visual indication of the warning condition.
- To verify the complete system using Wokwi simulation.

3. Technologies Used

- ESP32 development board
- MPU6050 accelerometer and gyroscope
- Arduino C/C++
- I2C communication
- arduinoFFT library
- Wokwi Simulator
- VS Code / Arduino project files
- Serial Monitor for output analysis

4. Methodology

The MPU6050 is connected to the ESP32 through the I2C interface. The ESP32 initializes the sensor and repeatedly requests the three-axis acceleration data. The raw values are converted to g units and a vibration magnitude is calculated from the deviation of the acceleration vector. A block of sampled data is then processed using the arduinoFFT library to obtain frequency-domain information and the dominant frequency is reported. The calculated vibration value is compared with a threshold. If the threshold condition is met, GPIO 4 drives the red LED and the serial monitor prints an abnormal-vibration warning.

5. Implementation

The project was implemented in a Wokwi ESP32 simulation using an MPU6050 sensor, a red LED and a 220 Ω resistor. The MPU6050 is powered from the ESP32 3.3 V supply and shares the ESP32 ground. SDA is connected to GPIO 21 and SCL to GPIO 22. The warning LED is controlled from GPIO 4 through the resistor.

The following connections were used:

- MPU6050 VCC $\rightarrow$ ESP32 3.3 V
- MPU6050 GND $\rightarrow$ ESP32 GND
- MPU6050 SDA $\rightarrow$ ESP32 GPIO 21
- MPU6050 SCL $\rightarrow$ ESP32 GPIO 22
- ESP32 GPIO 4 $\rightarrow$ 220 Ω resistor $\rightarrow$ LED anode
- LED cathode $\rightarrow$ ESP32 GND

The program uses 64 samples with a sampling frequency of 100 Hz for the FFT stage. The MPU6050 is addressed at 0x68. The serial monitor displays the X, Y and Z acceleration values, vibration level and, in the FFT version, the dominant frequency.

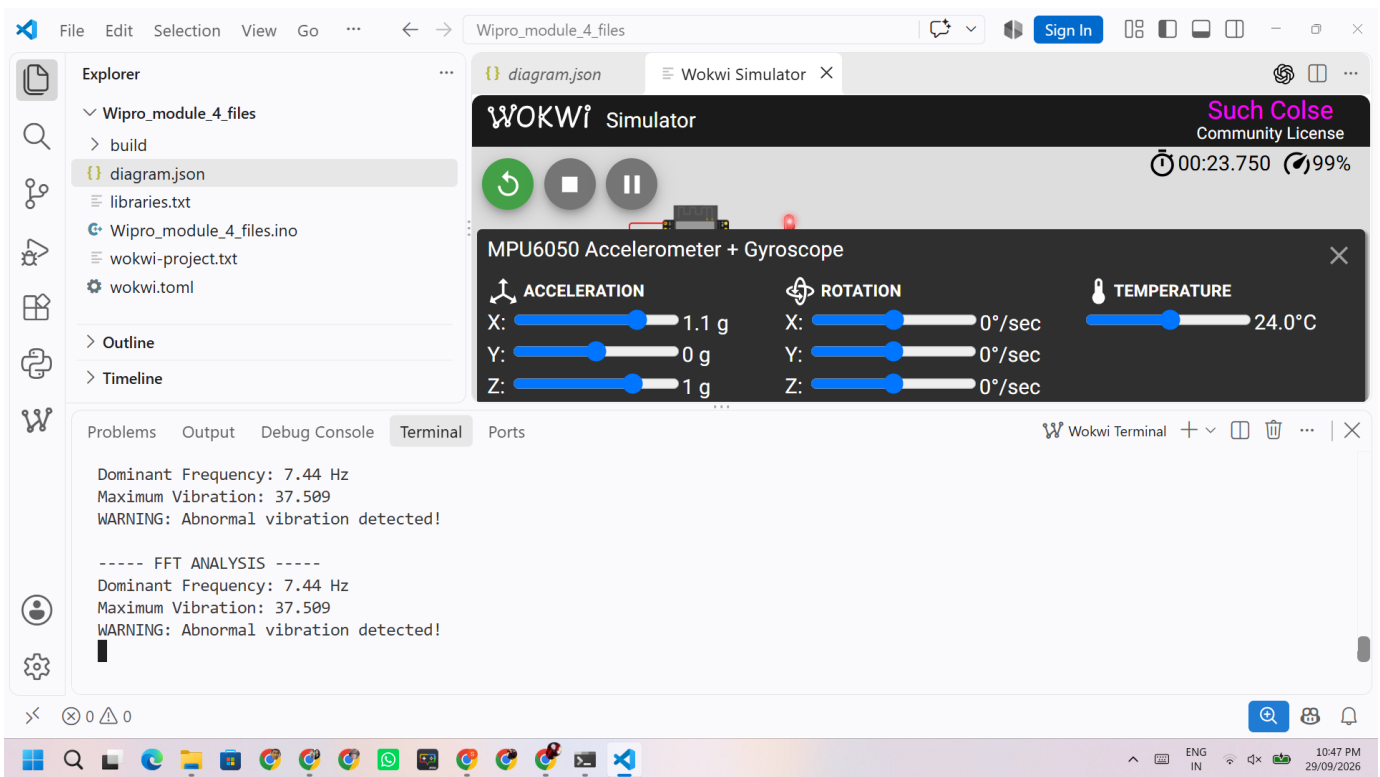


Figure 1: Final Wokwi/VS Code simulation view showing the ESP32, MPU6050, LED and serial FFT output. The serial monitor reports a dominant frequency of 7.44 Hz and an abnormal-vibration warning.

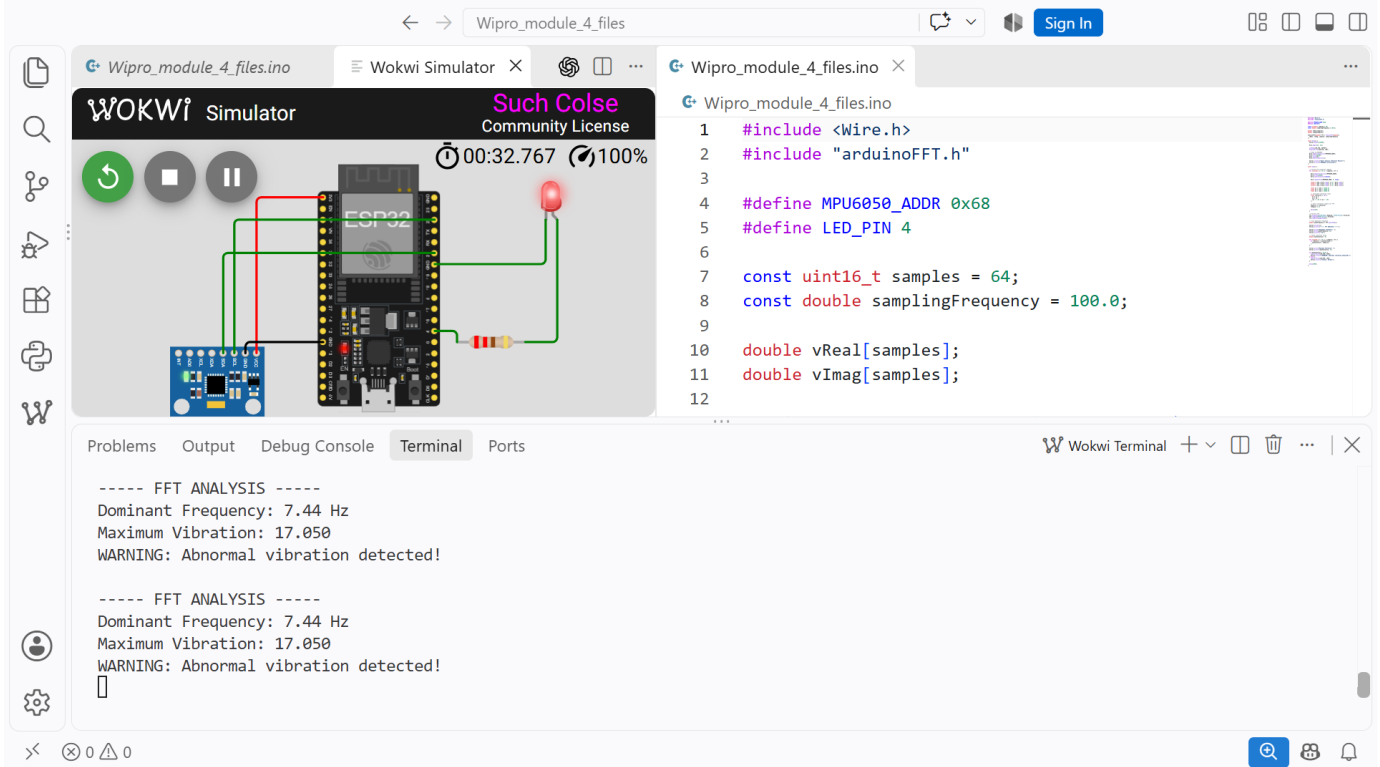


Figure 2: ESP32 and MPU6050 connected in the simulation with the warning LED illuminated. The serial output shows vibration analysis results.

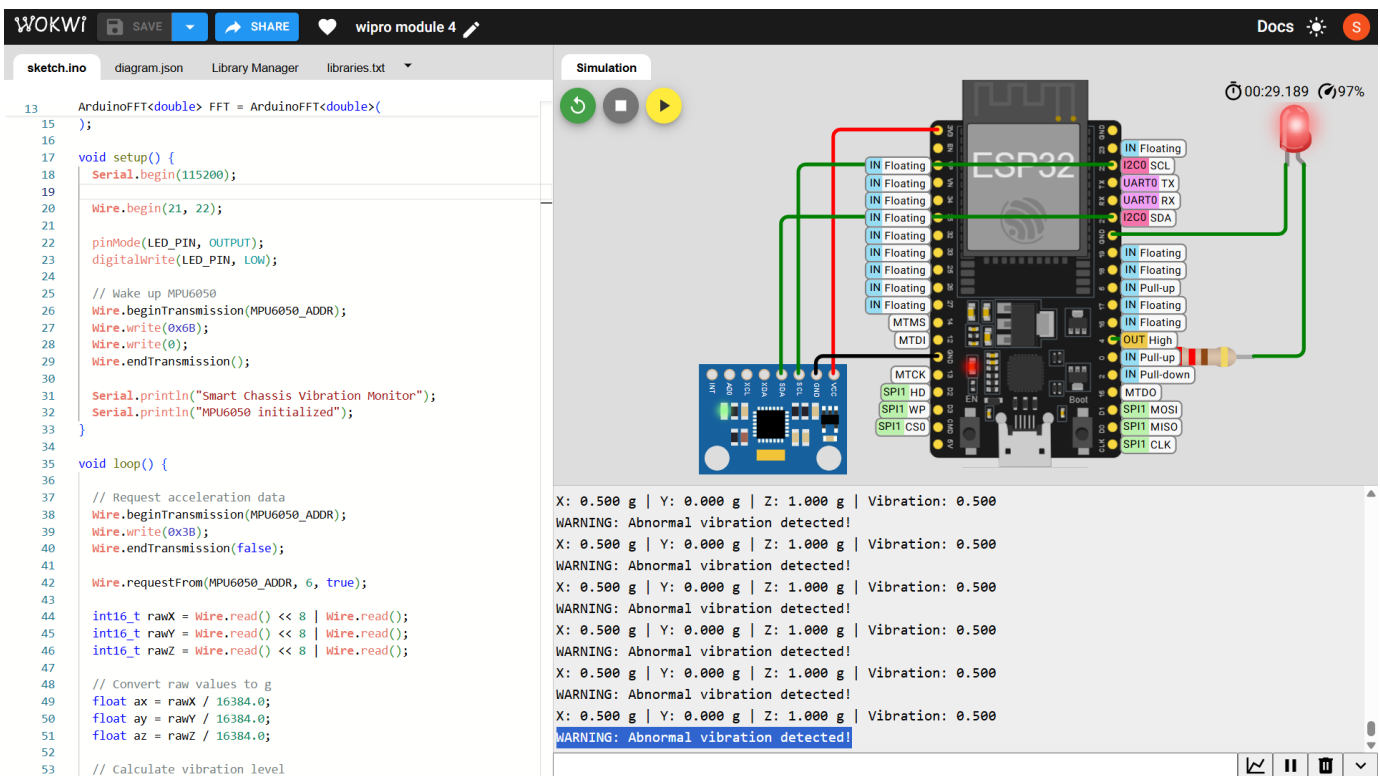


Figure 3: Complete Wokwi simulation with the source code visible and the red LED ON while abnormal vibration is reported in the serial monitor.

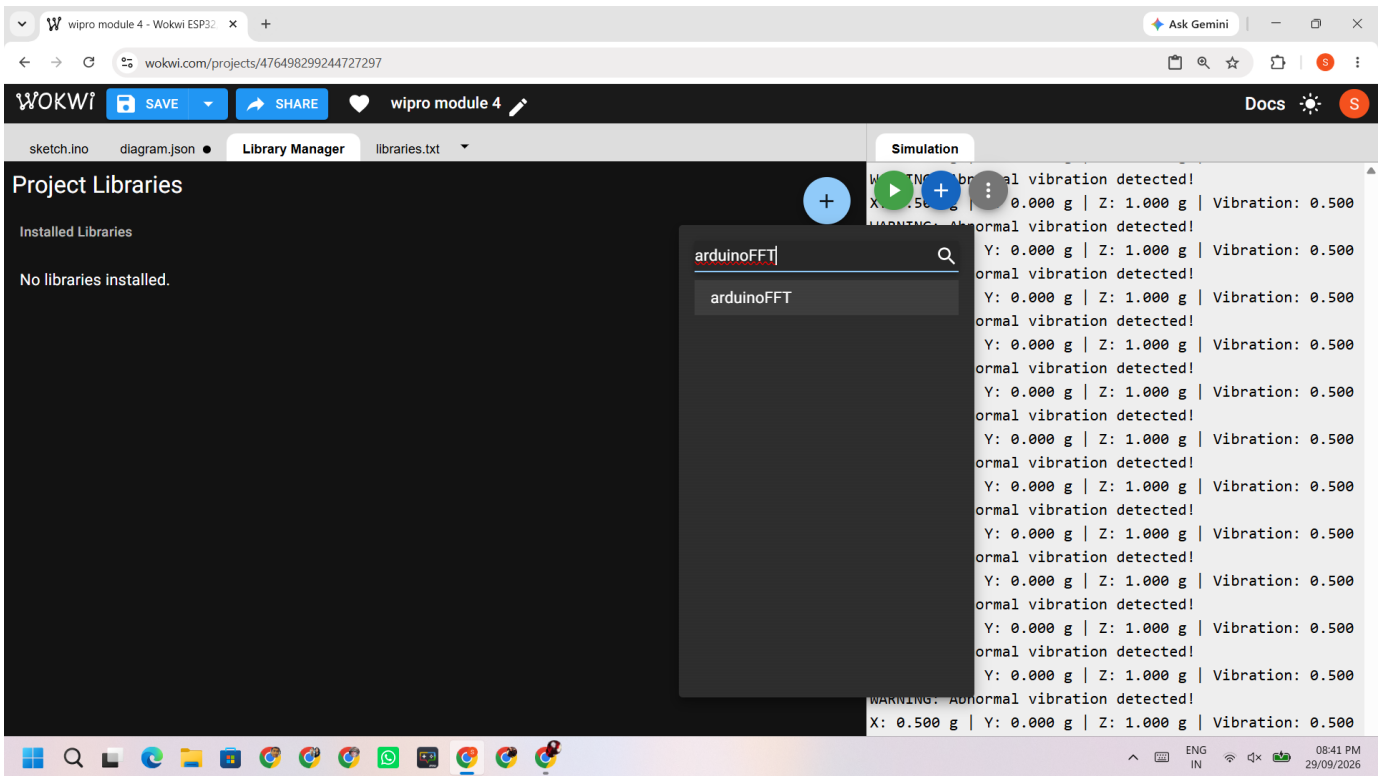


Figure 6: Searching for the arduinoFFT library in Wokwi's Library Manager.

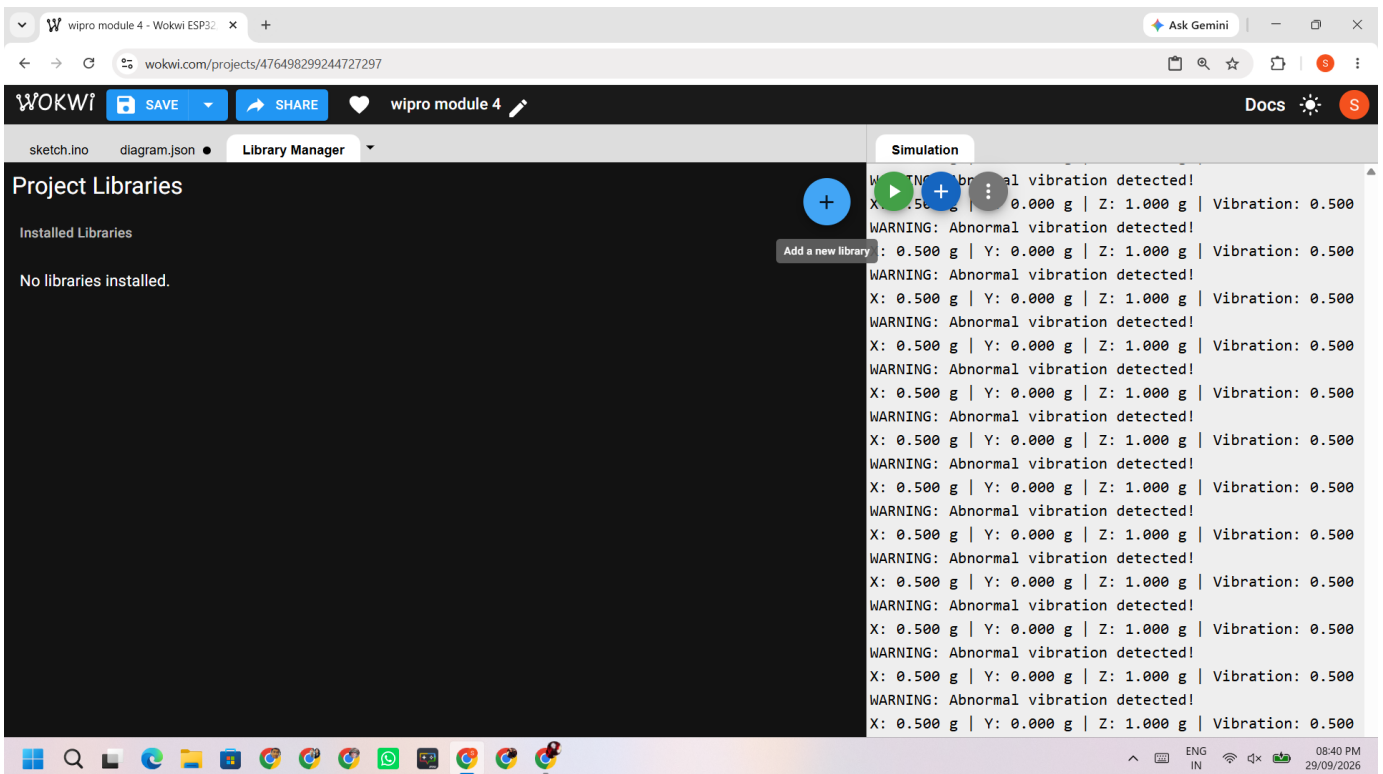


Figure 7: Library Manager view after selecting the arduinoFFT library for the project.

6. Results and Analysis

The system was successfully tested in Wokwi. The MPU6050 produced acceleration readings and the ESP32 processed them continuously. The FFT analysis produced a dominant frequency of approximately

7.44 Hz in the demonstrated run. The serial monitor also displayed abnormal-vibration warnings when the calculated vibration exceeded the programmed threshold.

- Dominant frequency observed in the FFT output: approximately 7.44 Hz.
- Maximum vibration values varied between simulation runs because the simulated sensor condition was not identical in every run.
- The serial monitor repeatedly displayed the abnormal-vibration warning during the warning condition.
- The red LED illuminated at the same time as the warning condition, providing direct visual evidence that the warning logic was active.
- The MPU6050 raw-data test confirmed communication between the sensor and the ESP32.

The LED-on screenshots are particularly important as proof of working operation: the simulated circuit is visible, the red LED is illuminated, and the serial monitor simultaneously reports the abnormal-vibration condition.

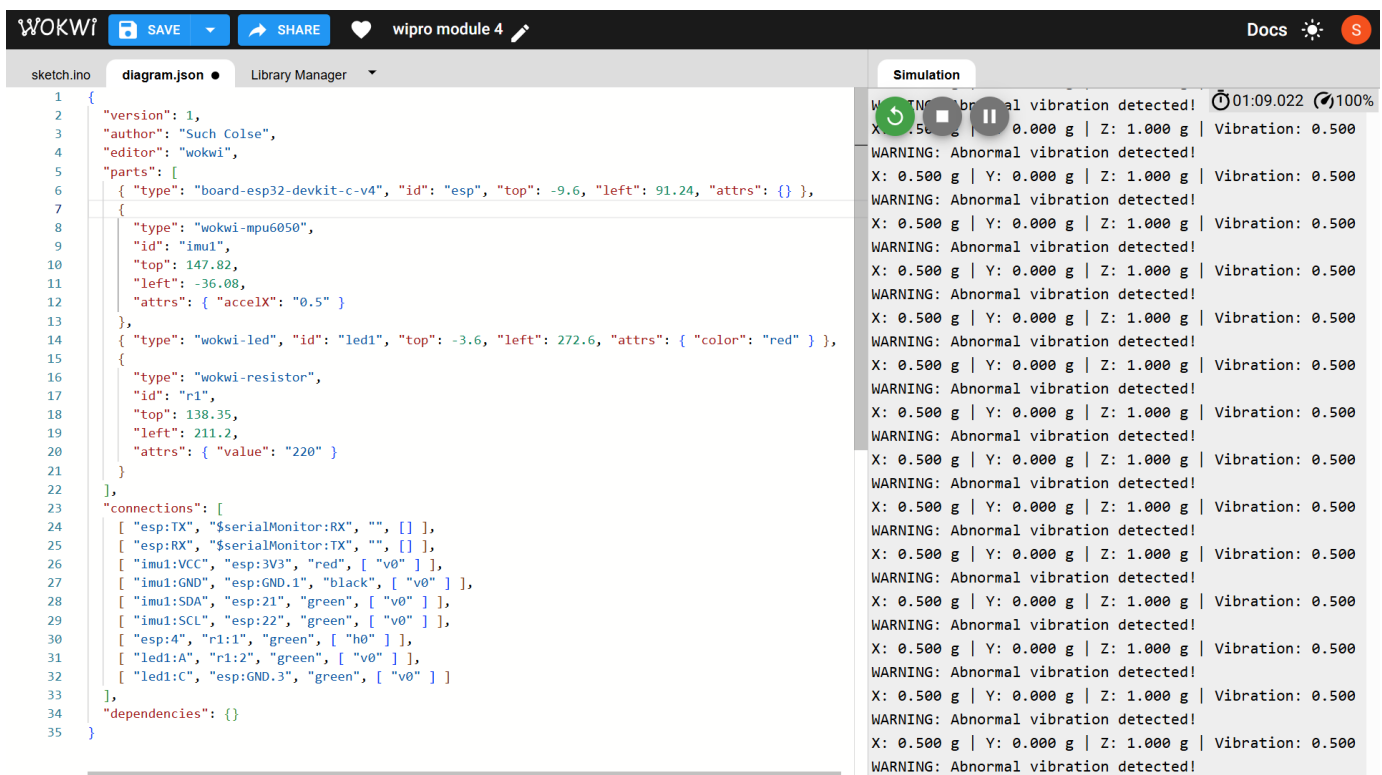


Figure 8: diagram.json showing the ESP32, MPU6050, red LED and 220-ohm resistor, together with the circuit connections.

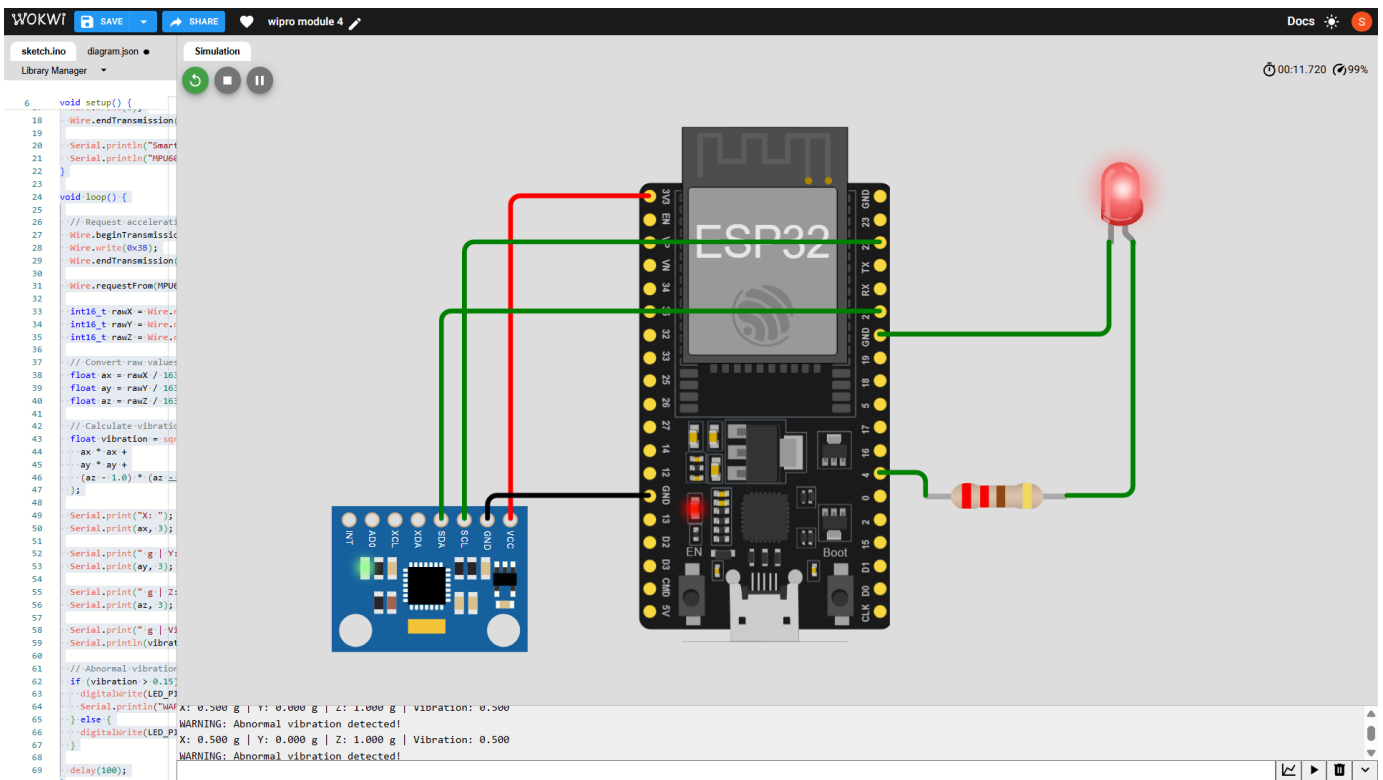


Figure 9: Wokwi simulation showing the complete circuit and the red warning LED illuminated.

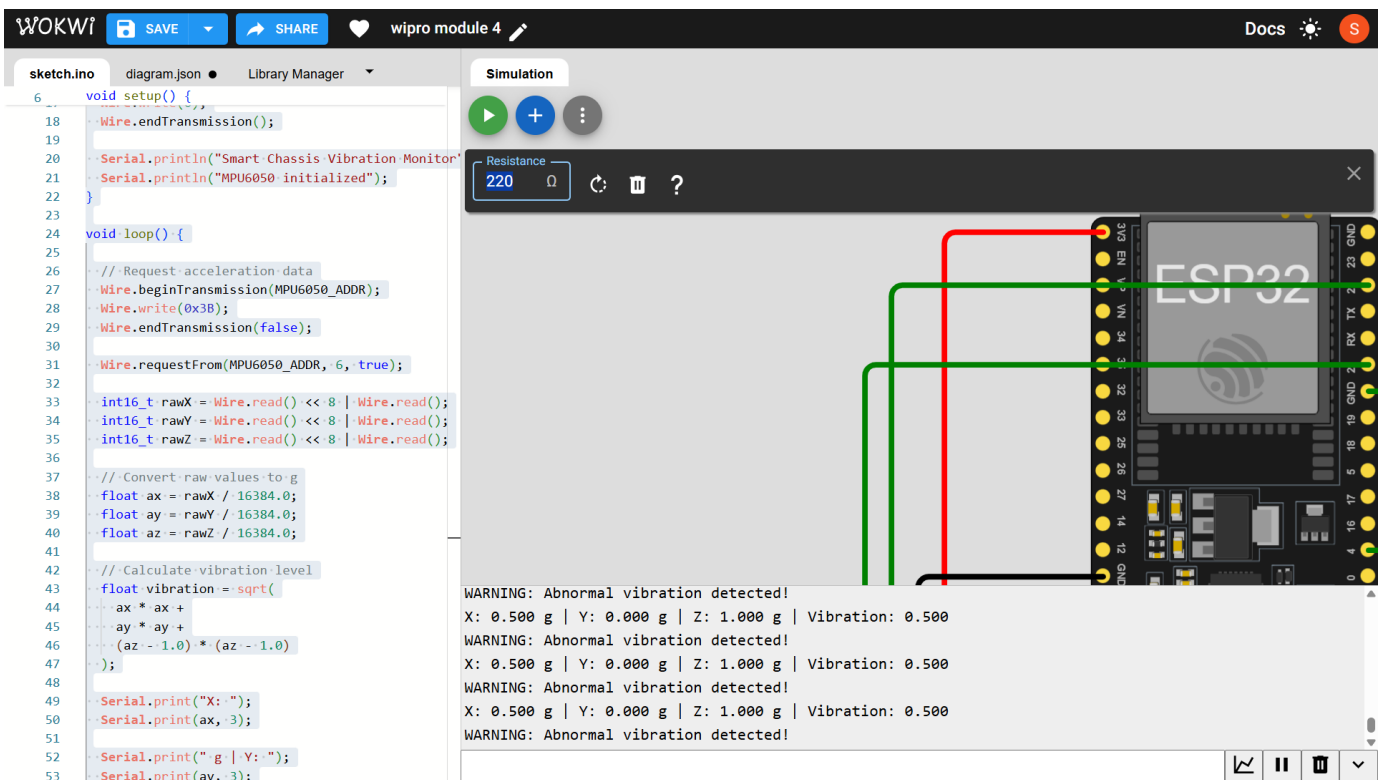


Figure 10: Source code and simulation view demonstrating the vibration calculation and warning logic with the LED ON.

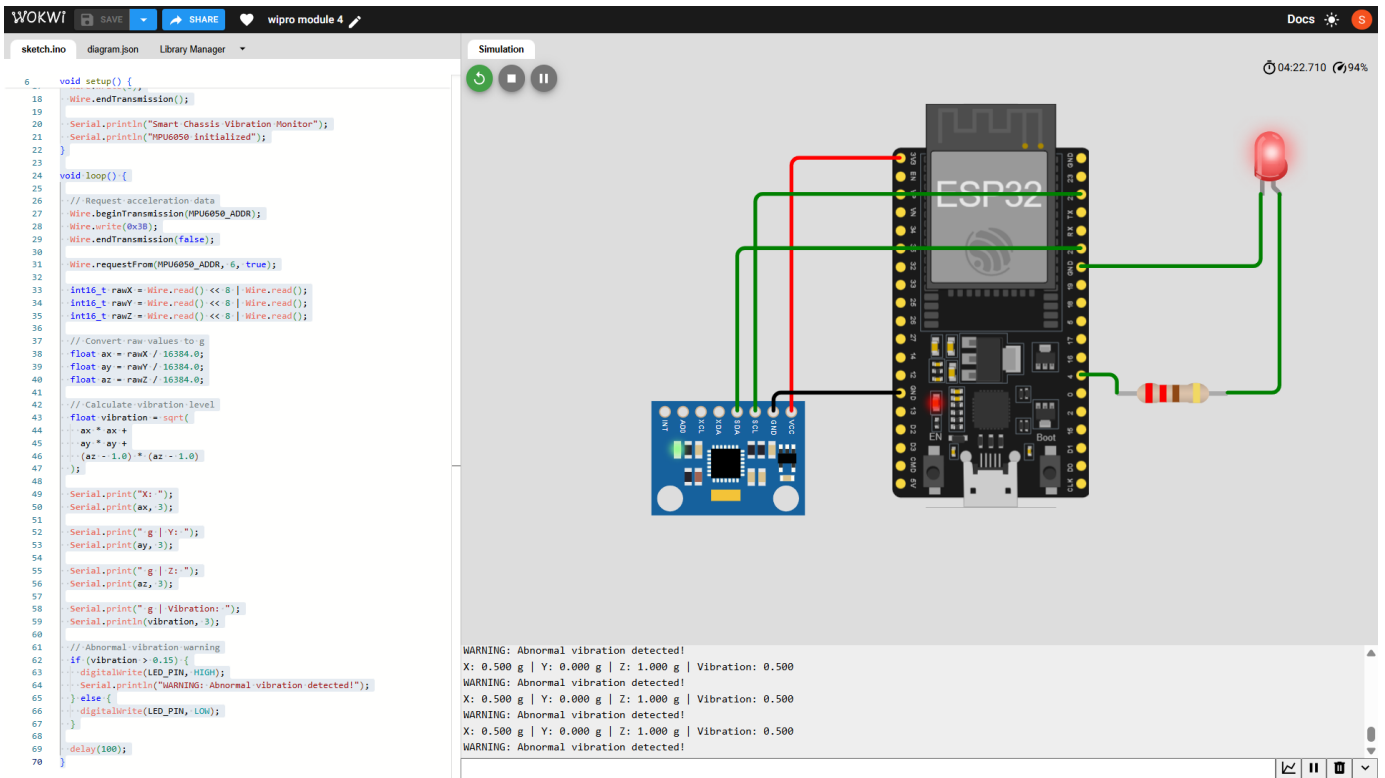


Figure 11: ESP32/MPU6050 simulation with the warning LED illuminated and serial vibration readings visible.

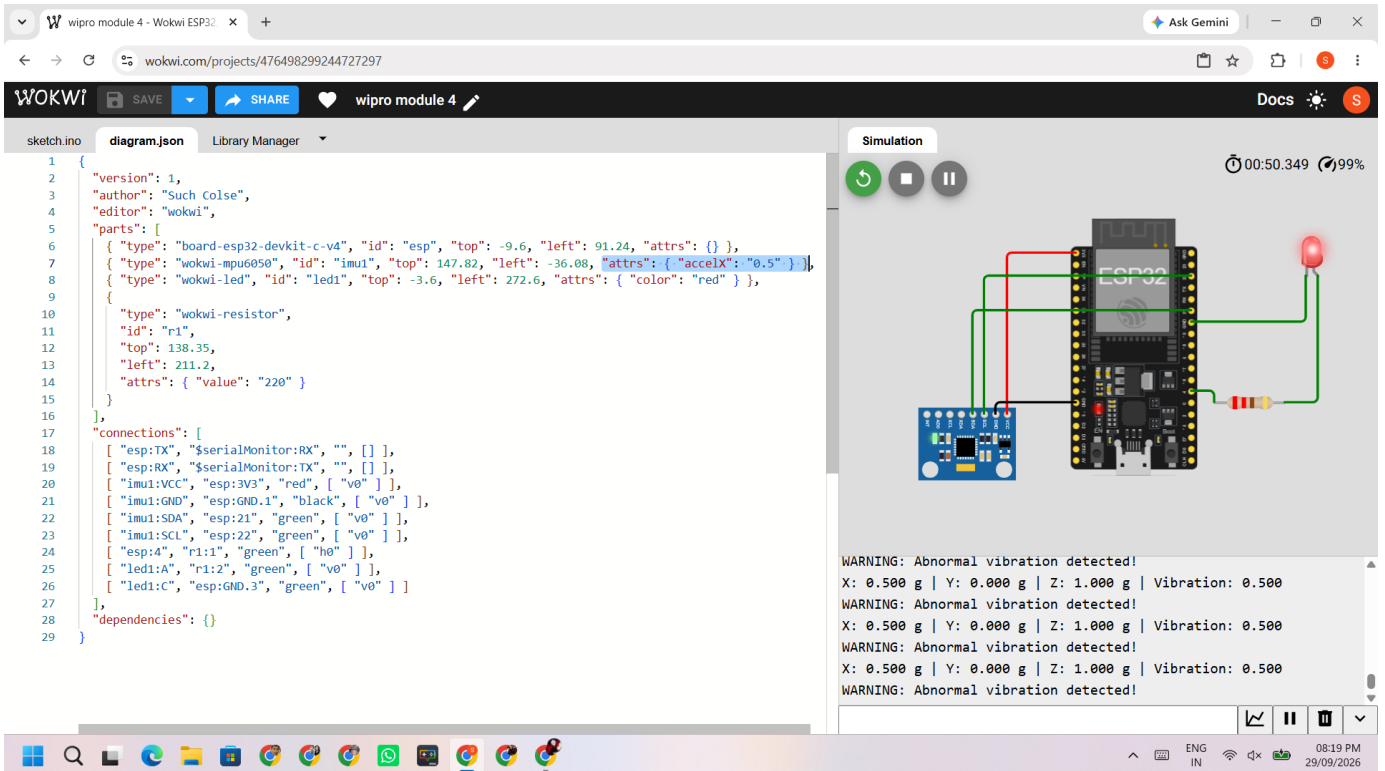


Figure 12: Source-code and simulation evidence of the vibration-monitoring logic and active warning indication.

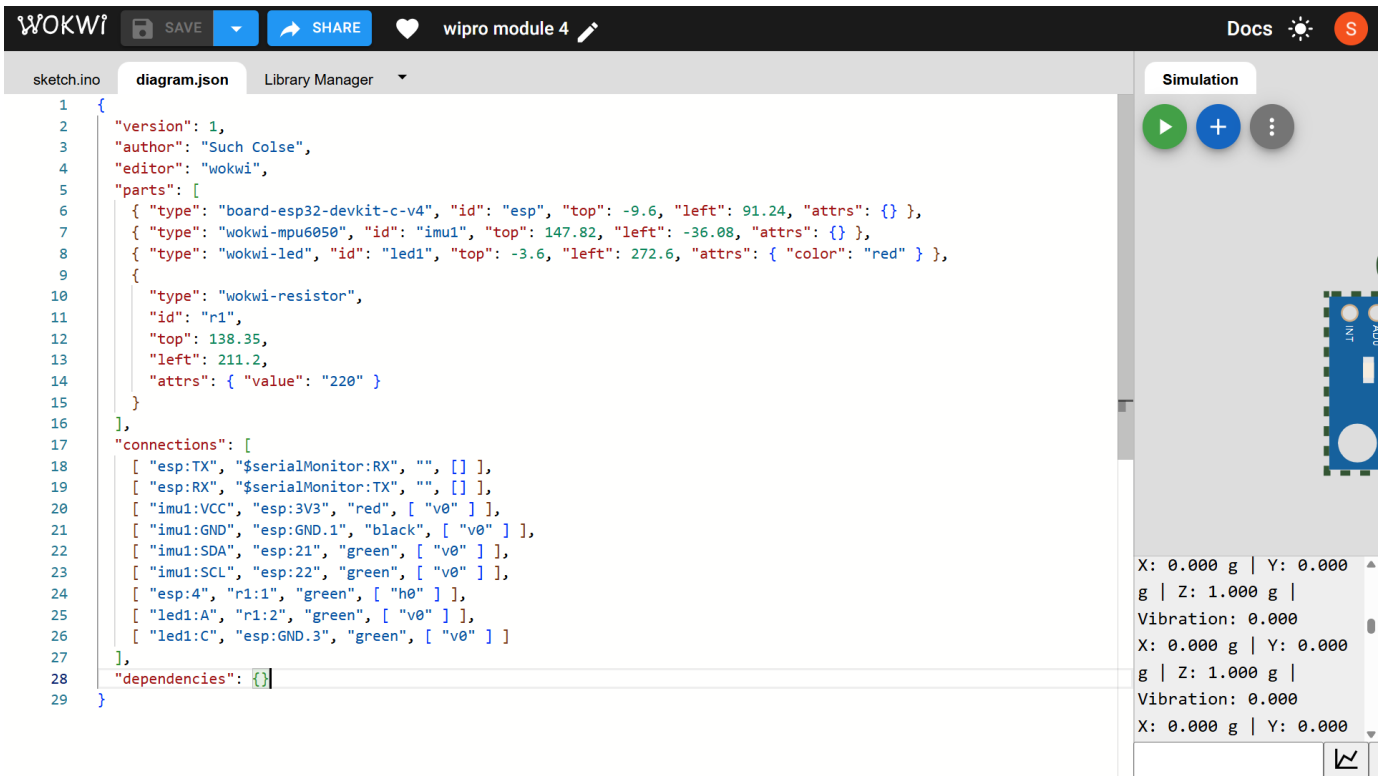


Figure 13: diagram.json and the simulated hardware arrangement used for the project.

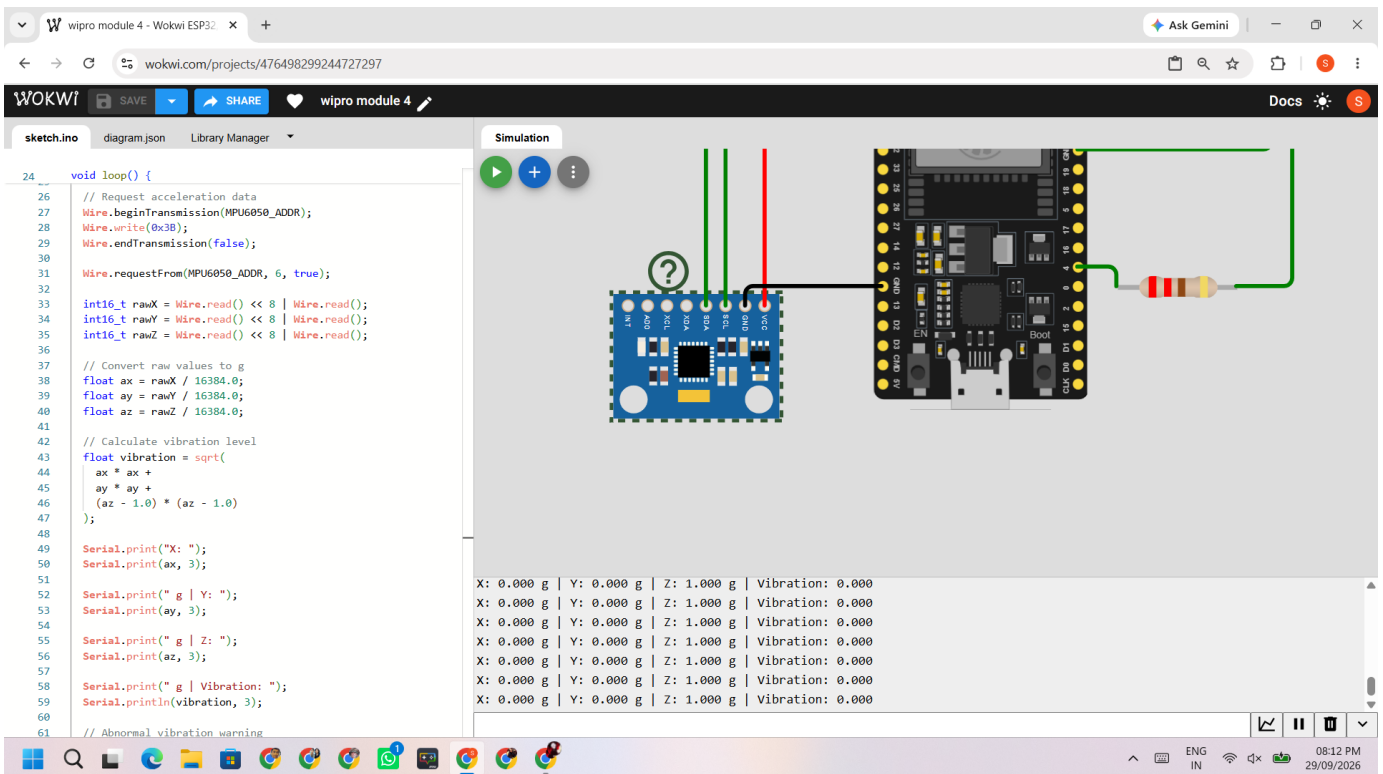


Figure 14: Simulation view showing the MPU6050 and ESP32 wiring with serial output during operation.

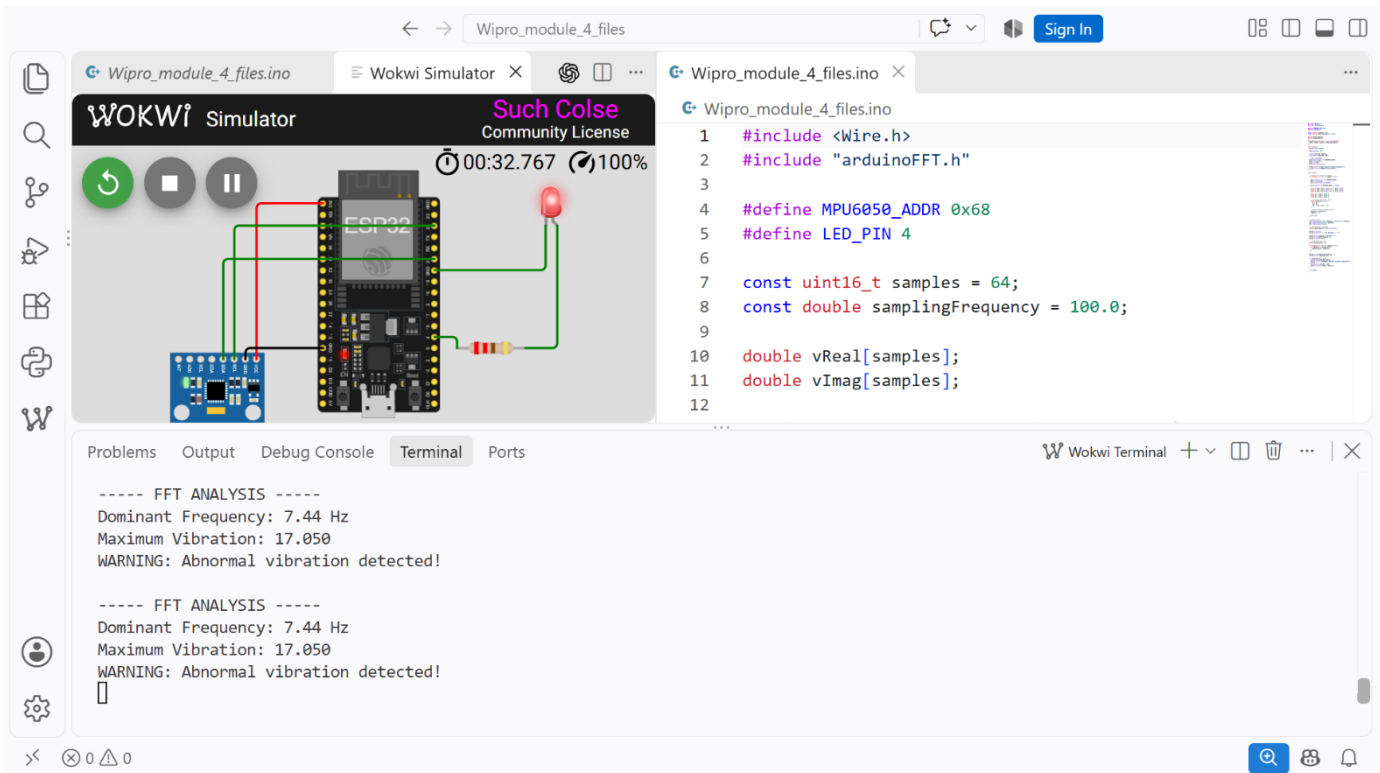


Figure 19: Final/extended simulation evidence with the circuit and serial readings visible.

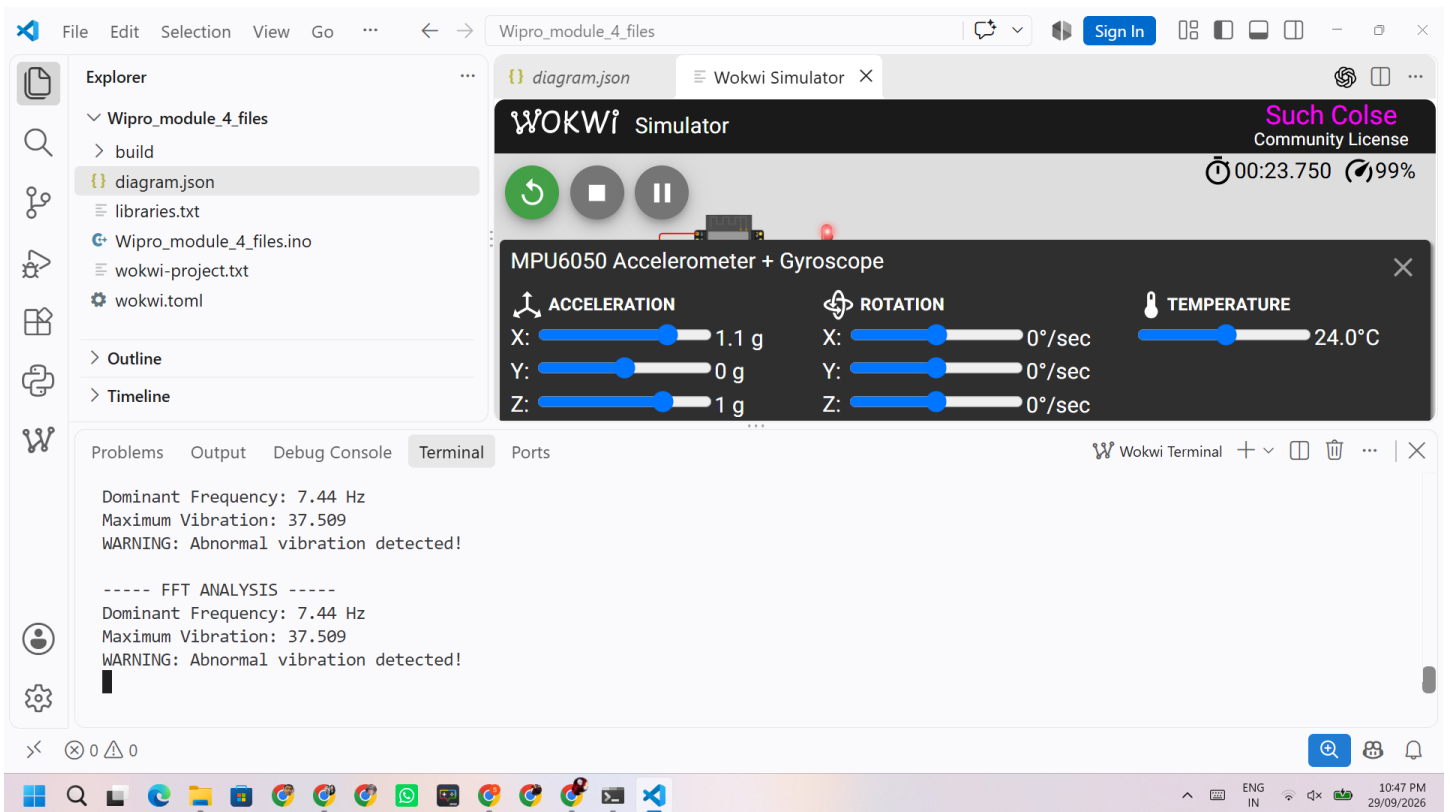


Figure 19: Final/extended simulation evidence with the Accelerometer and Gyroscope tweak values visible.

7. Conclusion

The Smart Chassis Vibration Analysis System successfully demonstrates vibration monitoring using an ESP32 and MPU6050. The system acquires acceleration data, calculates a vibration level, performs FFT-based frequency analysis and provides an abnormal-vibration warning. The Wokwi simulation screenshots show the circuit, sensor readings, FFT output and the red LED warning indication. Therefore, the prototype demonstrates the basic operation of an embedded chassis-vibration monitoring system.

8. Future Scope

- Add real-time vibration graphs and FFT plots.
- Store vibration history for predictive-maintenance analysis.
- Add Wi-Fi or Bluetooth transmission using the ESP32.
- Create a web dashboard for live vehicle-condition monitoring.
- Calibrate thresholds using real vehicle vibration measurements.
- Use machine-learning methods to classify different mechanical fault patterns.
- Integrate additional vehicle parameters such as temperature and engine RPM.

9. References

- Wokwi Documentation – ESP32 and simulation resources.
- arduinoFFT library documentation and examples.
- MPU6050 sensor documentation and register information.
- Arduino / ESP32 Wire (I2C) library documentation.

Wokwi Project:

<https://wokwi.com/projects/476498299244727297>

Source code:

```
#include <Wire.h>
#include "arduinoFFT.h"

#define MPU6050_ADDR 0x68
#define LED_PIN 4

const uint16_t samples = 64;
const double samplingFrequency = 100.0;

double vReal[samples];
double vImag[samples];

ArduinoFFT<double> FFT = ArduinoFFT<double>(
    vReal, vImag, samples, samplingFrequency
);
```

```

void setup() {
  Serial.begin(115200);

  Wire.begin(21, 22);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Wake up MPU6050
  Wire.beginTransmission(MPU6050_ADDR);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission();

  Serial.println("Smart Chassis Vibration Monitor");
  Serial.println("MPU6050 initialized");
}

void loop() {

  // Collect 64 vibration samples
  for (uint16_t i = 0; i < samples; i++) {

    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(0x3B);
    Wire.endTransmission(false);

    Wire.requestFrom(MPU6050_ADDR, 6, true);

    int16_t rawX = Wire.read() << 8 | Wire.read();
    int16_t rawY = Wire.read() << 8 | Wire.read();
    int16_t rawZ = Wire.read() << 8 | Wire.read();

    float ax = rawX / 16384.0;
    float ay = rawY / 16384.0;
    float az = rawZ / 16384.0;

    // Calculate vibration level
    float vibration = sqrt(
      ax * ax +
      ay * ay +
      (az - 1.0) * (az - 1.0)
    );

    // Store vibration sample for FFT
    vReal[i] = vibration;
    vImag[i] = 0;

    delay(10);
  }
}

```

```

// Perform FFT
FFT.windowing(FFTWindow::Hamming, FFTDirection::Forward);
FFT.compute(FFTDirection::Forward);
FFT.complexToMagnitude();

// Find dominant frequency
double peakFrequency = FFT.majorPeak();

Serial.println();
Serial.println("----- FFT ANALYSIS -----");

Serial.print("Dominant Frequency: ");
Serial.print(peakFrequency, 2);
Serial.println(" Hz");

// Check vibration level
double maxVibration = 0;

for (uint16_t i = 0; i < samples; i++) {
    if (vReal[i] > maxVibration) {
        maxVibration = vReal[i];
    }
}

Serial.print("Maximum Vibration: ");
Serial.println(maxVibration, 3);

if (maxVibration > 0.15) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("WARNING: Abnormal vibration detected!");
} else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("Status: Normal");
}

delay(500);
}

```

GitHub Repository Link:

(Here I have uploaded all the project files.)

https://github.com/parthibasarkar/Wipro_Smart_Chassis_Vibration_Analysis_System_module_4_files